

Project Report

24300720157 Ruijie Li

January 4, 2026

Contents

1	Introduction	2
2	Algorithm Description	2
2.1	Bidiagonalization	2
2.2	Divide-and-Conquer Strategy	3
2.3	Computing the SVD of M	4
2.3.1	Computing the Singular Vectors	5
2.3.2	Computing the Singular Values	6
2.4	Deflation	7
2.4.1	Local Deflation	7
2.4.2	Interblock Deflation	8
2.4.3	Global Deflation	9
3	Implementation Details	10
4	Numerical Experiments	11
5	Conclusion	14
6	Future Work	14

1 Introduction

The Singular Value Decomposition (SVD) is a cornerstone of numerical linear algebra, with applications spanning data compression, signal processing, and machine learning. While the standard Golub–Kahan algorithm is numerically stable, its sequential nature limits performance on modern parallel architectures. This led to the development of Divide-and-Conquer SVD algorithms (Cuppen, 1981), which recursively break the problem into subproblems, allowing for parallel execution and improved efficiency.

However, the naive implementation of the Divide-and-Conquer strategy suffers from a critical numerical instability. When singular values are closely spaced, the standard formulas for updating singular vectors are extremely sensitive to round-off errors, leading to significant inaccuracies in the computed SVD.

Fortunately, Gu and Eisenstat (1995) proposed a bidiagonal Divide-and-Conquer algorithm (BDC) that addresses these stability issues. Instead of directly computing the singular vectors from the original rank-1 perturbed system, their algorithm modifies the diagonal elements slightly to ensure that the secular equation’s roots are well-separated. This adjustment significantly enhances numerical stability, allowing for accurate computation of singular values and vectors even in challenging scenarios. The BDC algorithm computes all the eigenvalues in $O(n^2)$ time and the singular vectors in $O(n^3)$ time, making it efficient for large-scale problems.

This report presents a partial implementation of the Gu–Eisenstat Divide-and-Conquer SVD algorithm. In section 2, we provide a detailed description of the algorithm, highlighting the solving process of secular equation. Some important implementation details are discussed in section 3, including rational interpolation technique. Finally, section 4 showcases numerical experiments that validate the accuracy and efficiency of our implementation compared to Numpy’s built-in SVD function.

2 Algorithm Description

Given a matrix $A \in \mathbb{C}^{m \times n}$, the goal of the Divide-and-Conquer SVD algorithm is to compute its singular value decomposition $A = U\Sigma V^*$, where $U \in \mathbb{C}^{m \times m}$ and $V \in \mathbb{C}^{n \times n}$ are unitary matrices, and $\Sigma \in \mathbb{C}^{m \times n}$ is a diagonal matrix containing the singular values of A .

2.1 Bidiagonalization

The first step is to reduce A to a bidiagonal form using Householder transformations. This involves applying a series of unitary transformations from the left and right to zero out the appropriate elements, resulting in a matrix B such that $A = Q_1 B Q_2^*$, where Q_1 and Q_2 are unitary matrices and B is lower bidiagonal if $m \geq n$ or upper bidiagonal if $m < n$.

Since bidiagonalization is a standard, well-studied process with a complexity of $O(mn^2)$, it typically dominates the computational cost of the entire SVD. **To isolate and rigorously evaluate the performance of the Divide-and-Conquer algorithm (which specifically targets the bidiagonal phase), we will bypass this reduction step.** In our

implementation and experiments, we will generate random bidiagonal matrices B directly as inputs.

2.2 Divide-and-Conquer Strategy

The core idea of the BDC algorithm is to recursively divide the lower bidiagonal matrix $B \in \mathbb{R}^{(n+1) \times n}$ into smaller submatrices, compute their SVDs, and then merge the results to obtain the SVD of the original matrix. Given B , we divide B into two subproblems as follows:

$$B = \begin{bmatrix} B_1 & \alpha_k e_k & 0 \\ 0 & \beta_k e_1 & B_2 \end{bmatrix},$$

where $1 < k < n$, $B_1 \in \mathbb{R}^{k \times (k-1)}$ and $B_2 \in \mathbb{R}^{(n-k+1) \times (n-k)}$ are lower bidiagonal submatrices, and e_j is the j -th unit vector of appropriate dimension. The scalars α_k and β_k are the coupling elements that connect the two submatrices. Usually, we choose $k = \lfloor n/2 \rfloor$ to balance the sizes of the subproblems. We then recursively compute the SVDs of B_1 and B_2 and obtain:

$$B_i = [Q_i, q_i] \begin{bmatrix} D_i \\ 0 \end{bmatrix} W_i^T, \quad i = 1, 2.$$

Notice that

$$\begin{aligned} \alpha_k e_k &= [Q_1, q_1] \begin{bmatrix} Q_1^T \\ q_1^T \end{bmatrix} \alpha_k e_k = [Q_1, q_1] \begin{bmatrix} Q_1^T e_k \alpha_k \\ q_1^T e_k \alpha_k \end{bmatrix} \\ \beta_k e_1 &= [Q_2, q_2] \begin{bmatrix} Q_2^T \\ q_2^T \end{bmatrix} \beta_k e_1 = [Q_2, q_2] \begin{bmatrix} Q_2^T e_1 \beta_k \\ q_2^T e_1 \beta_k \end{bmatrix}. \end{aligned}$$

Let l_1^T and λ_1 be the last row and last element of Q_1 and q_1 , and similarly define f_2^T and φ_2 be the first row and first element of Q_2 and q_2 . We can rewrite B as

$$B = \begin{bmatrix} q_1 & Q_1 & & \\ & & Q_2 & q_2 \end{bmatrix} \begin{bmatrix} \boxed{\alpha_k \lambda_1} & & & \\ \alpha_k l_1 & D_1 & & \\ \beta_k f_2 & & D_2 & \\ \boxed{\beta_k \varphi_2} & & & \end{bmatrix} \begin{bmatrix} & W_1 & & \\ 1 & & & \\ & & W_2 & \end{bmatrix}^T$$

There is only one nonzero element in the first and last columns of the middle matrix. Applying a Givens rotation to zero out $\beta_k \varphi_2$, we have

$$\begin{aligned} B &= \begin{bmatrix} q_1 & Q_1 & & \\ & & Q_2 & q_2 \end{bmatrix} \begin{bmatrix} c & & & -s \\ & I_{k-1} & & \\ & & I_{n-k} & \\ s & & & c \end{bmatrix} \begin{bmatrix} r_0 & & & \\ \alpha_k l_1 & D_1 & & \\ \beta_k f_2 & & D_2 & \\ 0 & & & \end{bmatrix} \begin{bmatrix} & W_1 & & \\ 1 & & & \\ & & W_2 & \end{bmatrix}^T \\ &= \begin{bmatrix} (cq_1 & Q_1 & & \\ sq_2 & & Q_2 & \end{bmatrix} \begin{bmatrix} -sq_1 \\ cq_2 \end{bmatrix} \begin{bmatrix} r_0 & & & \\ \alpha_k l_1 & D_1 & & \\ \beta_k f_2 & & D_2 & \\ 0 & & & \end{bmatrix} \begin{bmatrix} & W_1 & & \\ 1 & & & \\ & & W_2 & \end{bmatrix}^T \\ &:= [Q \quad q] \begin{bmatrix} M \\ 0 \end{bmatrix} W^T, \end{aligned}$$

where c and s are the cosine and sine of the Givens rotation angle, and

$$r_0 = \sqrt{(\alpha_k \lambda_1)^2 + (\beta_k \varphi_2)^2}.$$

Thus, the problem reduces $\begin{bmatrix} M \\ 0 \end{bmatrix}$ by orthogonal transformations. Note that M has nonzero elements only on the diagonal and the first column. In next subsection, we will discuss how to compute the SVD of this matrix efficiently and stably. Now, suppose we have computed the SVD of M as $M = U\Omega V^T$, then the SVD of B can be constructed as

$$B = \begin{bmatrix} Q & q \end{bmatrix} \begin{bmatrix} U\Omega V^T \\ 0 \end{bmatrix} W^T := X \begin{bmatrix} \Omega \\ 0 \end{bmatrix} Y^T.$$

To compute the SVD of B_1 and B_2 , this process can be recursively applied until the submatrices are so small that their SVDs can be computed directly using Golub–Kahan algorithm or other stable methods. There can be at most $O(\log n)$ levels of recursion.

2.3 Computing the SVD of M

Now we focus on computing the SVD of the matrix

$$M = \begin{bmatrix} z_1 & & & \\ z_2 & d_2 & & \\ \vdots & & \ddots & \\ z_n & & & d_n \end{bmatrix}$$

where $D = \text{diag}(d_1, \dots, d_n)$, with $0 =: d_1 \leq d_2 \leq \dots \leq d_n$ and $z = [z_1, \dots, z_n]^T$. We further assume that

$$d_{j+1} - d_j \geq \tau \|M\|_2, \quad |z_j| \geq \tau \|M\|_2, \quad j = 1, \dots, n-1,$$

where τ is a small positive constant to be specified later. Any matrix of this form can be reduced to one that satisfies these conditions by using the deflation procedure described in the next subsection and a simple permutation.

Lemma 2.1. *Let $M = U\Omega V^T$ be the SVD of M with*

$$U = [u_1, \dots, u_n], \quad V = [v_1, \dots, v_n], \quad \Omega = \text{diag}(\omega_1, \dots, \omega_n),$$

Then the singular values ω_i satisfy the interlacing property

$$0 < \omega_1 < d_2 < \omega_2 < d_3 < \dots < d_n < \omega_n < d_n + \|z\|_2,$$

and the secular equation

$$f(\omega) := 1 + \sum_{k=1}^n \frac{z_k^2}{d_k^2 - \omega^2} = 0. \tag{1}$$

The singular vectors are given by

$$u_i = \begin{bmatrix} \frac{z_1}{d_1^2 - \omega_i^2} \\ \frac{z_2}{d_2^2 - \omega_i^2} \\ \vdots \\ \frac{z_n}{d_n^2 - \omega_i^2} \end{bmatrix} / \sqrt{\sum_{j=1}^n \left(\frac{z_j}{d_j^2 - \omega_i^2} \right)^2} \quad (2)$$

and

$$v_i = \begin{bmatrix} -1 \\ \frac{d_2 z_2}{d_2^2 - \omega_i^2} \\ \vdots \\ \frac{d_n z_n}{d_n^2 - \omega_i^2} \end{bmatrix} / \sqrt{1 + \sum_{j=2}^n \left(\frac{d_j z_j}{d_j^2 - \omega_i^2} \right)^2} \quad (3)$$

Given D and all singular values, we can reconstruct M using the lemma below.

Lemma 2.2. *Given a diagonal matrix $D = \text{diag}(d_1, \dots, d_n)$ and a set of numbers $\{\hat{\omega}_i\}_{i=1}^n$ satisfying the interlacing property*

$$0 = d_1 < \hat{\omega}_1 < d_2 < \hat{\omega}_2 < \dots < d_n < \hat{\omega}_n < d_n + \|z\|_2,$$

there exists a matrix

$$\hat{M} = \begin{bmatrix} \hat{z}_1 & & & \\ \hat{z}_2 & d_2 & & \\ \vdots & & \ddots & \\ \hat{z}_n & & & d_n \end{bmatrix}$$

whose singular values are exactly $\{\hat{\omega}_i\}_{i=1}^n$. The elements $\hat{z}_i$ can be computed by

$$|\hat{z}_i| = \sqrt{(\hat{\omega}_n^2 - d_i^2) \prod_{k=1}^{i-1} \frac{(\hat{\omega}_k^2 - d_i^2)}{(d_k^2 - d_i^2)} \prod_{k=i}^n \frac{(\hat{\omega}_k^2 - d_i^2)}{(d_{k+1}^2 - d_i^2)}} \quad (4)$$

You can refer to Gu and Eisenstat (1995) for the proofs of Lemma 2.1 and 2.2.

2.3.1 Computing the Singular Vectors

In practice, we can compute the approximate singular values $\{\hat{\omega}_i\}_{i=1}^n$ by solving the secular equation (1). Once we have the approximate singular values, we can compute the singular vectors using equations (2) and (3). However, directly using these formulas can lead to numerical instability, especially when the singular values are closely spaced — small perturbations can cause great loss of orthogonality among the computed singular vectors. To address this issue, Gu and Eisenstat (1995) proposed a more stable method for computing the singular vectors.

Lemma 2.2 allows us to reconstruct a matrix $\hat{M}$ whose singular values are exactly the computed approximate singular values $\{\hat{\omega}_i\}_{i=1}^n$. We can then compute the singular vectors of $\hat{M}$ using equations (2) and (3), which are more stable since the singular values are now exact. This approach significantly improves the numerical stability of the singular vector computation. Thus, the orthogonality of the computed singular vectors can be guaranteed to high precision. We can use $\hat{M}$'s SVD as an approximation to M 's SVD.

2.3.2 Computing the Singular Values

Now we discuss how to compute the singular values by solving the secular equation (1). However, if we directly compute $d_k^2 - \omega^2$, it may lead to severe cancellation errors when ω is close to d_k . To avoid this, we can compute the offsets. Specifically, for any $i \leq n-1$, we first check whether $f\left(\frac{d_i+d_{i+1}}{2}\right)$ is positive to determine whether the root ω_i lies in the left or right half of the interval (d_i, d_{i+1}) .

First assume that $\omega_i \in \left(d_i, \frac{d_i+d_{i+1}}{2}\right)$. Now assume the offset $\mu = \omega_i - d_i$ and the length of the interval $\delta_j = d_j - d_i$. Then the denominator can be computed as

$$d_j^2 - \omega_i^2 = (d_j - d_i - \mu)(d_j + d_i + \mu) = (\delta_j - \mu)(d_i + d_j + \mu).$$

Note that this method is numerically stable since $\delta_j - \mu \in (\delta_j/2, \delta_j)$. We will then seek the root $\mu_i = \omega_i - d_i \in (0, \delta_{i+1}/2)$ in the reformulated secular equation

$$f(\mu_i) := 1 + \sum_{k=1}^n \frac{z_k^2}{(\delta_k - \mu_i)(d_k + d_i + \mu_i)} = 0.$$

For the convenience of subsequent discussion, we denote

$$\begin{aligned} \varphi_i(\mu) &= \sum_{k=1}^i \frac{z_k^2}{(\delta_k - \mu)(d_k + d_i + \mu)}, \\ \psi_i(\mu) &= \sum_{k=i+1}^n \frac{z_k^2}{(\delta_k - \mu)(d_k + d_i + \mu)}. \end{aligned}$$

Now assume that $\omega_i \in \left(\frac{d_i+d_{i+1}}{2}, d_{i+1}\right)$. We set the offset $\mu_i = d_{i+1} - \omega_i$ and the length of the interval $\delta_j = d_j - d_{i+1}$. We can similarly give the reformulated secular equation

$$f(\mu_i) := 1 + \sum_{k=1}^n \frac{z_k^2}{(\delta_k + \mu_i)(d_k + d_{i+1} - \mu_i)} = 0.$$

We will seek the root $\mu_i = d_{i+1} - \omega_i \in (0, \delta_i/2)$. Similarly, we denote

$$\begin{aligned} \varphi_i(\mu) &= \sum_{k=1}^i \frac{z_k^2}{(\delta_k + \mu)(d_k + d_{i+1} - \mu)}, \\ \psi_i(\mu) &= \sum_{k=i+1}^n \frac{z_k^2}{(\delta_k + \mu)(d_k + d_{i+1} - \mu)}. \end{aligned}$$

Finally we consider the last singular value $\omega_n > d_n$. Let $\mu_n = \omega_n - d_n$ and $\delta_j = d_j - d_n$, and we have the secular equation

$$f(\mu_n) := 1 + \sum_{k=1}^n \frac{z_k^2}{(\delta_k - \mu_n)(d_k + d_n + \mu_n)} = 0.$$

And we will seek the root $\mu_n \in (0, \|z\|_2)$.

We can use bisection or the rational interpolation method to solve these secular equations efficiently. The stopping criterion is set to be

$$|f(\mu)| \leq n\varepsilon(1 + |\varphi_i(\mu)| + |\psi_i(\mu)|),$$

2.4 Deflation

2.4.1 Local Deflation

Let $\tau = 2n^2\varepsilon$. Now we will show that we can reduce M to a same form that satisfies

$$|d_i - d_j| \geq \tau\|M\|_2, \quad \forall i \neq j, \quad |z_i| \geq \tau\|M\|_2.$$

First, we will discuss how to handle the case when some z_i is very small. There are two subcases:

1. If $|z_1| < \tau\|M\|_2$, then we will replace M by

$$M = \begin{bmatrix} z_1 & & & \\ z_2 & d_2 & & \\ \vdots & & \ddots & \\ z_n & & & d_n \end{bmatrix} = \begin{bmatrix} \tau\|M\|_2 & & & \\ & z_2 & d_2 & \\ & \vdots & & \ddots \\ & z_n & & & d_n \end{bmatrix} + O(\tau\|M\|_2).$$

The perturbed matrix has the same structure as M and satisfies the condition $|z_1| \geq \tau\|M\|_2$.

2. If $|z_i| < \tau\|M\|_2$, for some $i \geq 2$, then we will replace M by

$$M = \begin{bmatrix} z_1 & & & \\ z_2 & d_2 & & \\ \vdots & & \ddots & \\ z_n & & & d_n \end{bmatrix} = \begin{bmatrix} z_1 & & & \\ \vdots & \ddots & & \\ 0 & & d_i & \\ \vdots & & & \ddots \\ z_n & & & & d_n \end{bmatrix} + O(\tau\|M\|_2).$$

In this case, the i -th singular value of M is exactly d_i by Gershgorin circle theorem, and the corresponding left and right singular vectors are e_i . Thus, we can deflate this singular value and reduce the problem size by 1. The principal submatrix maintains the same structure as M .

Next, we will discuss how to handle the case when some d_i and d_j are very close. There are two subcases:

1. If $|d_i - d_1| = |d_i| < \tau\|M\|_2$ for $i \geq 2$, we can set d_i to zero and zero out the corresponding z_i . Specifically, we can apply a Givens rotation on the rows 1 and i to zero out z_i :

$$GM = \begin{bmatrix} c & & s & & \\ & I & & & \\ -s & & c & & \\ & & & I & \end{bmatrix} \begin{bmatrix} z_1 & & & \\ z_2 & d_2 & & \\ \vdots & & \ddots & \\ z_n & & & d_n \end{bmatrix} = \begin{bmatrix} r & & & & \\ z_2 & d_2 & & & \\ \vdots & & \ddots & & \\ 0 & & & 0 & \\ \vdots & & & & \ddots \\ z_n & & & & & d_n \end{bmatrix} + O(\tau\|M\|_2).$$

Thus, the i -th singular value of M is exactly zero, and the corresponding left singular vector is $G^T e_i$ and right singular vector is e_i . We can deflate this singular value and reduce the problem size by 1. The principal submatrix maintains the same structure as M .

2. If $|d_j - d_i| < \tau \|M\|_2$, similarly, we can apply contract Givens rotations on rows i and j to zero out z_j :

$$\begin{aligned}
 GMG^T &= \begin{bmatrix} I & & & \\ & c & & s \\ & & I & \\ & -s & & c \\ & & & I \end{bmatrix} \begin{bmatrix} z_1 & & & \\ z_2 & d_2 & & \\ \vdots & & \ddots & \\ z_n & & & d_n \end{bmatrix} \begin{bmatrix} I & & & \\ & c & & -s \\ & & I & \\ & s & & c \\ & & & I \end{bmatrix} \\
 &= \begin{bmatrix} z_1 & & & & & \\ \vdots & \ddots & & & & \\ r & & d_j & & & \\ \vdots & & & \ddots & & \\ 0 & & & & d_j & \\ \vdots & & & & & \ddots \\ z_n & & & & & & d_n \end{bmatrix} + O(\tau \|M\|_2).
 \end{aligned}$$

In the perturbed matrix, the j -th singular value is exactly d_j by Gershgorin circle theorem, and the leading principal submatrix maintains the same structure as M .

This process is called deflation of M by Gu and Eisenstat, but I think it is more appropriate to call it local deflation since it only considers the elements within M .

2.4.2 Interblock Deflation

Note: The following two sections describe the standard strategy proposed by Gu and Eisenstat. In our current simplified implementation, we focus primarily on Local Deflation.

In the dividing strategy, we write

$$B = \begin{bmatrix} Q & q \end{bmatrix} \begin{bmatrix} M \\ 0 \end{bmatrix} W^T,$$

where

$$Q = \begin{bmatrix} c_0 q_1 & Q_1 \\ s_0 q_2 & Q_2 \end{bmatrix}, \quad M = \begin{bmatrix} r_0 & & \\ \alpha_k l_1 & D_1 & \\ \beta_k f_2 & & D_2 \end{bmatrix}, \quad W = \begin{bmatrix} & W_1 \\ 1 & \\ & & W_2 \end{bmatrix}.$$

Note that both Q and W are sparse. Thus, if $M = U\Sigma V^*$, then the main cost is to compute the matrix multiplication QU and WV . Gu proposed a deflation strategy to reduce the cost of these multiplications.

If r_0 is small or $(\alpha_k l_1^T, \beta_k f_2^T)$ has some small elements, we can apply the deflation procedure described in the previous subsection without changing the block structure of Q and W .

If D_1 has a small element or several close elements, we can also apply the deflation procedure to D_1 , and similarly for D_2 . Again, these cases won't change the block structure of Q and W .

But if there are some close elements between D_1 and D_2 , and we apply the deflation procedure above, then the block structure of Q and W will be destroyed. In this case, we can apply a permutation to M . Gu proposed a "polarized permutation" strategy. We will move one of the close elements to the end of D_2 and the other to the start of D_1 . This process is just like polarization. Like previous cases, we can apply Givens rotation to zero out the last element of $\beta_k f_2$:

$$GMG^T = \begin{bmatrix} r_0 & & & & \\ r & d_n & & & \\ \alpha_k l'_1 & & D'_1 & & \\ \beta_k f'_2 & & & D'_2 & \\ 0 & & & & d_n \end{bmatrix} + O(\tau \|M\|_2) = \begin{bmatrix} M' & \\ & d_n \end{bmatrix} + O(\tau \|M\|_2).$$

As a cost, this deflation will still change the block structure of Q and W . However, since we have placed the close elements at the boundary of D_1 and D_2 , we can still maintain most of the sparsity of Q and W . Specifically, we have

$$\begin{aligned} B &= [QG^T \quad q] \begin{bmatrix} GMG^T \\ 0 \end{bmatrix} (WG^T)^T \\ &= [X_1 \quad x \quad q] \begin{bmatrix} M' & \\ & d_n \\ 0 & 0 \end{bmatrix} [Y_1 \quad y]^T + O(\tau \|M\|_2), \end{aligned}$$

where

$$X_1 = \begin{bmatrix} c_0 q_1 & c q'_1 & Q'_1 & 0 \\ s_0 q_2 & s q'_2 & 0 & Q'_2 \end{bmatrix}, \quad x = \begin{bmatrix} -s q'_1 \\ c q'_2 \end{bmatrix}$$

and

$$Y_1 = \begin{bmatrix} 0 & c w'_1 & W'_1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & s w'_2 & 0 & W'_2 \end{bmatrix}, \quad y = \begin{bmatrix} -s w'_1 \\ 0 \\ c w'_2 \end{bmatrix}.$$

d_n is an approximate singular value that can be deflated. The corresponding left singular vector is x and right singular vector is y . The matrix M_1 has the same structure as M and can be deflated in a similar fashion until no diagonal element of D_1 is close to a diagonal element of D_2 .

2.4.3 Global Deflation

We look back to the whole BDC algorithm. To illustrate global deflation, we look at two levels of the dividing strategy.

$$B = \begin{bmatrix} B_1 & \alpha_{i+j} e_{i+j} & 0 \\ 0 & \beta_{i+j} e_1 & B_2 \end{bmatrix} = \begin{bmatrix} B_{1,1} & \alpha_i e_i & 0 & 0 & 0 \\ 0 & \beta_i e_1 & B_{1,2} & \alpha_{i+j} e_j & 0 \\ 0 & 0 & 0 & \beta_{i+j} e_1 & B_2 \end{bmatrix}$$

Let $X_{1,2} \begin{bmatrix} D_{1,2} \\ 0 \end{bmatrix} Y_{1,2}^T$ be the SVD of $B_{1,2}$, and let $(f_{1,2}^T, \varphi_{1,2})$ and $(l_{1,2}^T, \lambda_{1,2})$ be the first and last rows of $\tilde{X}_{1,2}$, respectively. Then

$$B = \tilde{X} \begin{bmatrix} B_{1,1} & \alpha_i e_i & 0 & 0 & 0 \\ 0 & \beta_i f_{1,2} & D_{1,2} & \alpha_{i+j} l_{1,2} & 0 \\ 0 & \beta_i \varphi_{1,2} & 0 & \alpha_{i+j} \lambda_{1,2} & 0 \\ 0 & 0 & 0 & \beta_{i+j} e_1 & B_2 \end{bmatrix} \tilde{Y}^T,$$

Let $\bar{d}_s$ be the s th diagonal element of $D_{1,2}$, and let $\bar{f}_s$ and $\bar{l}_s$ be the s th components of $f_{1,2}$ and $l_{1,2}$, respectively. Then ignoring all zero components, the $(i+s)$ th column and row of the middle matrix are $(\bar{d}_s)$ and $(\beta_i \bar{f}_s, \bar{d}_s, \alpha_{i+j} \bar{l}_s)$, respectively. Thus if both $|\beta_i \bar{f}_s|$ and $|\alpha_{i+j} \bar{l}_s|$ are small, then we can perturb them to zero. $\bar{d}_s$ is a singular value of the perturbed matrix and the $(i+s)$ th columns of $\tilde{X}$ and $\tilde{Y}$ are the corresponding left and right singular vectors, respectively. This singular value and its singular vectors can be deflated from all subsequent subproblems. We call this global deflation.

3 Implementation Details

My code structure is as follows:

```

1 src/
2 |-- __init__.py
3 |-- dnc_svd.py
4 |-- secular.py
5 |-- utils.py
6 tests/
7 |-- test_secular.py
8 report/
9 |-- report.tex
10 |-- report.pdf

```

Listing 1: Core Project Structure Snippet

In `dnc_svd.py`, I implemented the BDC algorithm. In `secular.py`, I implemented the secular equation solver with local deflation. There is no need to talk about `dnc_svd.py`, since the main content has been discussed in previous sections. What I want to highlight is the implementation of `secular.py`. In this part, we use rational interpolation to solve the secular equation. More precisely, we use fractional forms to approximate the secular function $f(\omega)$:

$$f(\omega) \approx \frac{C_1}{\omega - d} + C_2.$$

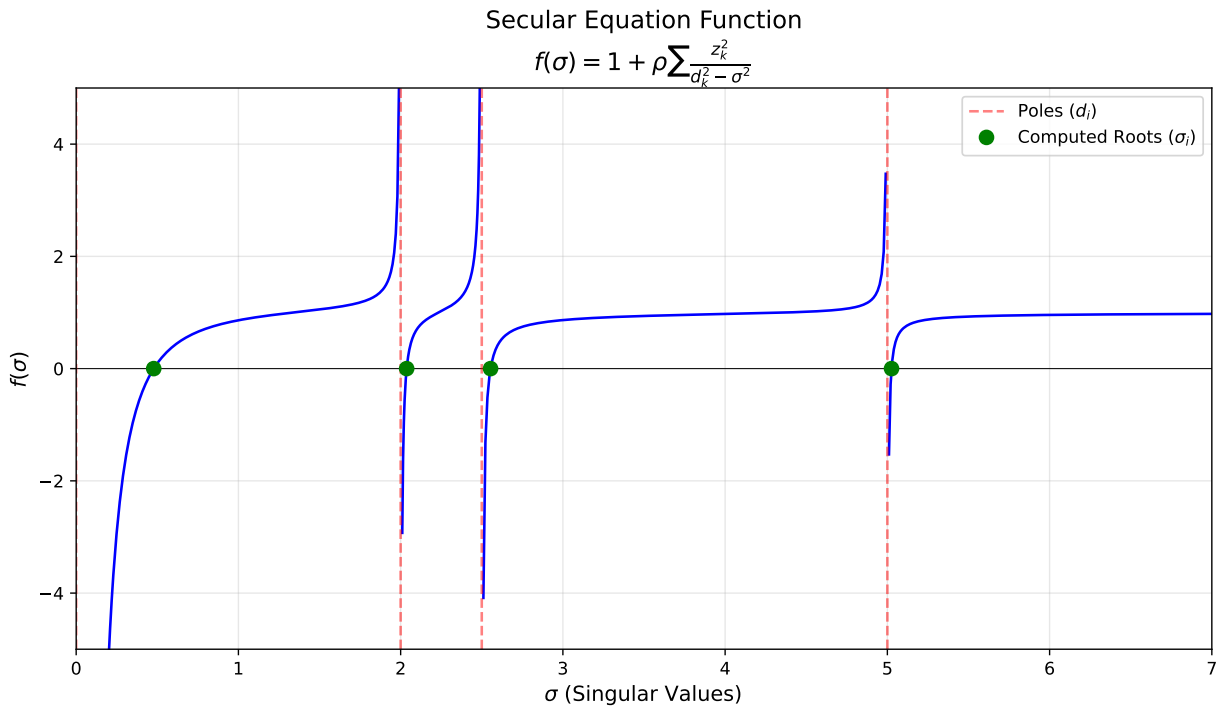
Here, d is the nearest pole to the current approximation of the root. C_1 and C_2 can be computed by evaluating $f(\omega)$ and its derivative at the current approximation. Then we can compute the next approximation by solving the equation

$$\frac{C_1}{\omega - d} + C_2 = 0.$$

This method converges very fast when the initial approximation is good. But sometimes it can lead to points outside the interval. To avoid this, we combine **rational interpolation** and **bisection** together. In most cases, we use rational interpolation. If the next approximation is outside the interval, we use bisection to get the next approximation.

4 Numerical Experiments

Sorry that my implementation of the BDC algorithm is full of bugs and I haven't fixed them all yet. Thus, I cannot provide numerical experiments for BDC now. But, I have tested the secular equation solver with local deflation. The results are shown below.



The above figure shows the secular function $f(\omega)$ for an arrowhead matrix. The green dots are the computed singular values. As we can see, the secular function is very close to zero at these points, which indicates that our secular equation solver works well. This experiment can be seen in `exp_secular.py` in experiments folder.

Another example of the secular equation solver is shown below:

```

Mask: [False False  True False]
Roots: [0.77756016 2.47621455 2.          4.15496832]
Givens Rotations: 1
  > Checking orthogonality of U: ||U^T * U - I||_F = 4.041689e-16
  > Checking orthogonality of V: ||V^T * V - I||_F = 4.146804e-16
  > Reconstruction Error (MM^T): 1.94e-15
.
=== Test 2: Deflation Case (Small z Component) ===
Mask: [False  True False]
Roots: [0.94351882 2.          3.1795868 ]
  > Checking orthogonality of U: ||U^T * U - I||_F = 2.489914e-16
  > Checking orthogonality of V: ||V^T * V - I||_F = 1.396073e-16
  > Reconstruction Error (MM^T): 2.65e-16
.
=== Test 4: Permutation Consistency ===
[[ 0.00366419  0.01191723 -0.99992227]
 [-0.99997198 -0.00648331 -0.00374164]
 [ 0.0065274  -0.99990797 -0.01189314]]
.
=== Test 1: Standard Case (No Deflation) ===
Roots: [0.4778654  2.06192819 3.04468337]
  > Checking orthogonality of U: ||U^T * U - I||_F = 2.727887e-16
  > Checking orthogonality of V: ||V^T * V - I||_F = 3.203331e-16
  > Reconstruction Error (MM^T): 2.08e-16
.
-----
Ran 5 tests in 0.025s

OK

```

Figure 1: Numerical experiment for secular equation solver. The test has five different phases. The first one is standard case. The second one has two small z elements. The third one has two close d elements. The fourth one intentionally creates unsorted d elements to test the permutation. The last one tests the general numerical accuracy by comparing $f(\omega)$ with zero at computed singular values.

You can see this experiment in `test_secular.py` in test folder.

In fact, I have tested the main solver `DCSVDSolver` in `dnc_svd.py` as well, but there exist huge errors in the computed singular values. I don't know why. I have turned to AI-s, but none of them can help me fix the bugs. The results are shown below:

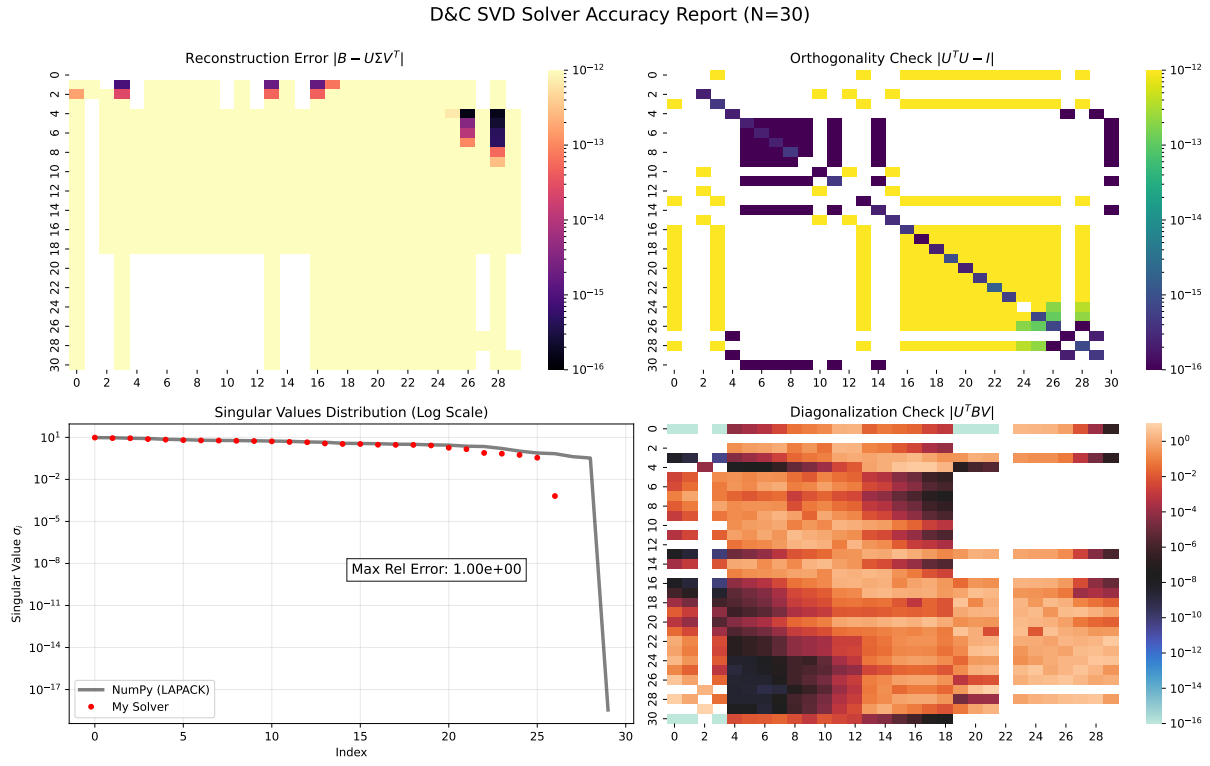


Figure 2: Numerical experiment for BDC algorithm.

We can observe that the main error occurs in the largest singular values. The problem can't be in the secular equation solver, since the same size of problem can be solved accurately there. I suspect that the problem lies in the dividing strategy or the merge strategy. But I have no idea how to fix it now. I will keep working on it in the future.

Although the BDC algorithm is not working now, the main framework is ready. So we can test the runtime of the algorithm. You can see the results below:

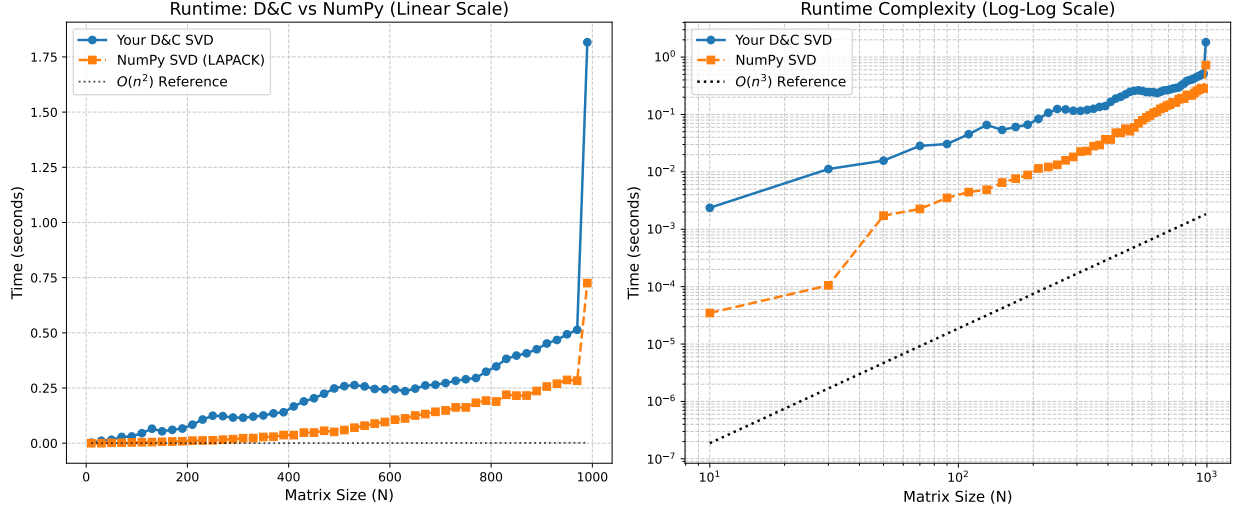


Figure 3: Runtime experiment for BDC algorithm.

Although the theoretical worst-case complexity for computing singular vectors is $O(n^3)$, our experiments exhibit a runtime complexity closer to $O(n^2)$. This empirically demonstrates the effectiveness of the deflation strategy implemented in our solver, which significantly reduces the dimensionality of the secular equations for the test matrices. But you can still see that there exist an increase of slope when the matrix size goes to 1000. This indicates that the algorithm is actually $O(n^3)$.

5 Conclusion

In this report, we have discussed the divide-and-conquer algorithm for computing the singular value decomposition of a matrix. We have explained the main ideas of the algorithm, including the dividing strategy, the merge strategy, and the secular equation solver with local deflation. We have also provided some implementation details and numerical experiments.

6 Future Work

There are several directions for future work:

1. Fix the bugs in the BDC algorithm implementation and ensure its correctness.
2. Implement interblock deflation and global deflation to improve the efficiency of the algorithm.
3. Conduct more extensive numerical experiments to evaluate the performance of the BDC algorithm compared to other SVD algorithms.